

AADK 6 - Task 2 — Designing Interactive Screen with Multiple Buttons & States

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 6

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1 Chosen Feature Idea

Feature: 🌙 *Theme Toggle + Counter Reset Screen*

The screen will allow users to:

- Toggle between **Light Mode** and **Dark Mode**
- Increase a counter
- Reset the counter

This feature uses **multiple buttons** and **multiple states**, making it ideal for interaction analysis.

2 Screen Layout Plan (UI Structure)

Layout Components:

- Title: *“Interactive Demo Screen”*
- Text showing:
 - Current Theme (Light/Dark)
 - Counter Value
- Buttons:
 - **Toggle Theme**
 - **Increase Count**
 - **Reset**

Layout Structure (Column-based Compose UI)

Interactive Demo Screen

Current Theme: Light

Counter Value: 0

[Toggle Theme]

[Increase Count]

[Reset]

This layout works well on both phone and tablet using a centered Column.

3 Button Actions & State Changes

◆ State Variables

```
var isDarkTheme by remember { mutableStateOf(false) }
```

```
var count by remember { mutableStateOf(0) }
```

○ Button A — Toggle Theme

Action:

```
isDarkTheme = !isDarkTheme
```

State Change:

- false → true (Light → Dark)
- true → false (Dark → Light)

UI Feedback:

- Background color changes
 - Text updates to show current theme
-

○ Button B — Increase Count

Action:

count++

State Change:

- Counter increases by 1

UI Feedback:

- Text updates: Counter Value: X
-

○ Button C — Reset

Action:

count = 0

State Change:

- Counter resets to zero

UI Feedback:

- Text shows Counter Value: 0
-

🔑 State Table (Required State Representation)

Current State	Trigger	Button	Resulting UI Change
Light Mode, Count = 0	Toggle Theme		Switch to Dark Mode (background + text update)
Dark Mode, Count = 3	Toggle Theme		Switch to Light Mode
Any Theme, Count = X	Increase Count		Counter increases to X+1
Any Theme, Count > 0	Reset		Counter becomes 0
Dark Mode, Count = 5	Reset		Count resets, theme remains Dark

This table shows at least **3+ distinct states** and how each button affects them.

5 How UI Responds (State Flow Explanation)

- Buttons trigger **state updates**.
 - Compose detects changes in `isDarkTheme` or `count`.
 - Only composables that read those states recompose.
 - Background, text, and counter update automatically.
-

6 Reflection (4–5 Lines)

If state is not managed cleanly, the UI may show inconsistent values, such as incorrect counter numbers or theme not matching the state. Poor state separation can also cause unnecessary recompositions and performance issues. To avoid this, I will follow state hoisting principles and separate UI from logic using `ViewModel` when needed. I will test UI transitions by rotating the device, resizing the emulator, and checking behavior on both phone and tablet layouts.